

Project 2



ID511001: Programming 2

Assessment Information

Level	Credits	Assessment Type	Weighting
5	15	Individual	35%

Assessment Overview

In this individual assessment, you will design and develop a Guess Who game as a Windows Forms Application using C#.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Build interactive, event-driven GUI applications using pre-built components.
2. Declare and implement user-defined classes using encapsulation, inheritance and polymorphism.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Project 1	35%	10 May at 11.59 PM	1, 2
Project 2	35%	21 June at 11.59 PM	1, 2
Skills-Based Assessment	30%	21st May from 1:00 PM – 2:45 PM / 19th May from 3:00 PM – 4:45 PM	1, 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by 21 June at 11.59 PM.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of 50% across all assessments in ID511001: Programming 2.

Submission

You must submit all application files via GitHub Classroom.

- Repository URL: http://classroom.github.com/a/Ury_53GW
- Git Ignore: If you do not have one, create a .gitignore using this resource - [VisualStudio.gitignore](#)
- Due Date: Sunday at 11.59 PM
- Late Penalty: 10% per day, rolling over at 12.00 AM

The latest application files in the main branch will be used to mark against the marking rubric. Please test your applications before you submit. Partial marks may be given for incomplete functionality.

Authenticity

All parts of your submitted assessment must be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

AI Tools

Learning to use AI tools is an important skill. While AI tools are powerful, you must be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You must still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository README.md file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

This also applies to code snippets retrieved from StackOverflow and GitHub.

Failure to do this may result in a mark of zero for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with Otago Polytechnic policies. Learners can view policies on the Otago Polytechnic website located at <https://www.op.ac.nz/about-us/governance-and-management/policies>.

Extensions

Familiarise yourself with the assessment due date. Extensions will only be granted if you are unable to complete the assessment by the due date because of unforeseen circumstances outside your control. The length of the extension granted will depend on the circumstances and must be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in ID511001: Programming 2.

Game Overview

Guess Who is a two-player deduction game. Each player selects a secret character from a shared pool. Players take turns asking yes/no questions to eliminate characters until one player correctly guesses the other's secret character.

Your application should implement a **human vs. computer** version of the game. The computer opponent should ask questions and eliminate characters automatically based on the responses.

Instructions

You will need to submit an application and documentation that meet the following requirements.

Functionality - Learning Outcomes 1, 2 (60%)

Part 1 - Data Model (5%)

- **Attribute.cs** - A class representing a single character attribute, e.g. hair colour, eye colour, or gender. This class should have fields for **name** and **value**.
- **Character.cs** - A class representing a Guess Who character. This class should have fields for **name**, a **List<Attribute>** of attributes, and a **bool** indicating whether the character has been eliminated. Each character must have at least **five attributes**, e.g. gender, hair colour, eye colour, has hat, or has glasses.
- **CharacterSeeder.cs** - A **static** class with a static method **SeedCharacters()** that returns a **List<Character>** containing at least **20 characters**, each with their full set of attributes. Each attribute combination should be unique enough that questions can eliminate characters effectively.

Hint: think carefully about your attributes. Good attributes are binary, i.e., Yes/No, or have a small number of distinct values so that each question eliminates a meaningful number of characters.

Part 2 - Game Logic (10%)

- **Question.cs** - A class representing a yes/no question the player can ask. This class should have fields for **text**, i.e., the question string, e.g. "Does your character have blue eyes?",

`attributeName`, and `attributeValue`. It should have a method `Matches(Character character)` that returns `true` if the character's attribute matches the question's target value.

- **QuestionSeeder.cs** - A `static` class with a static method `SeedQuestions()` that returns a `List<Question>` covering every attribute in the character pool. Every attribute value that appears in your character set should have a corresponding question.
 - **GameBoard.cs** - A class representing the state of one player's board. This class should have:
 - A `List<Character>` of all characters
 - A `Character` property for the board's secret character
 - A method `EliminateCharacters(Question question, bool answer)` that eliminates all characters that do not match the answer
 - A method `GetRemainingCharacters()` that returns only non-eliminated characters
 - A method `IsOnlyOneRemaining()` that returns `true` when exactly one character has not been eliminated
-

Part 3 - Players and Serialisation (10%)

- **Player.cs** - An abstract base class with fields for `name` and a `GameBoard`. It should have an abstract method `TakeTurn()`.
- **HumanPlayer.cs** - Inherits from `Player`. Overrides `TakeTurn()` to allow the user to select a question from the UI and receive a yes/no answer.
- **ComputerPlayer.cs** - Inherits from `Player`. Overrides `TakeTurn()` to automatically select the most useful question, i.e., the one that eliminates the most characters, and apply the answer.
- **GameRecord.cs** - A class representing a completed game's outcome. This class should have a `string` field for `playerName`, a `bool` field for `won`, an `int` field for `totalTurns`, and a `DateTime` field for `dateTime`.
- **GameHistory.cs** - A `static` class responsible for serialising and deserialising the list of `GameRecord` objects to and from a JSON file, e.g. `game_history.json`, using `System.Text.Json`. It should have methods `Save(List<GameRecord> records)` and `Load()` that return a `List<GameRecord>`.

Hint: `GameRecord` must have a public parameterless constructor and public properties for `System.Text.Json` to serialise it correctly. Refer to the Week 09 lecture notes for the save/load pattern.

Part 4 - Windows Forms UI (15%)

- **Form1.cs** - The main game form. This class should implement the following functionality:
 - Seed all characters and questions by calling the static seeder methods in the form constructor. **[0.5 mark]**
 - Display all characters on the human player's board as a grid of `Button` or `PictureBox` controls. Each control should show the character's name. **[1 mark]**
 - Allow the human player to visually eliminate characters by clicking on them, e.g. toggling a greyed-out or crossed-out state. **[1 mark]**
 - Display the current list of remaining non-eliminated characters on a `ListBox` or `Label`. **[0.5 mark]**

- Allow the human player to select a question from a **ComboBox** populated with all available questions. **[1 mark]**
 - When the human player asks a question, query the computer player's secret character and display the answer - Yes or No - in a **Label**. Eliminate the appropriate characters from the board. **[1.5 marks]**
 - Implement the computer player's turn: automatically select and display the question the computer asks, prompt the human player for a Yes/No answer using **Button** controls, and eliminate characters from the computer's board accordingly. **[2 marks]**
 - Allow the human player to make a final guess by selecting a character name. If correct, display a win message; if incorrect, display a loss message. **[1.5 marks]**
 - Allow the computer player to make a final guess when only one character remains on its board. If correct, the computer wins; display an appropriate message. **[1 mark]**
 - At the end of each game, save a **GameRecord** to **game_history.json** using **GameHistory**. **[1 mark]**
 - Display all past game records in a **DataGridView** on a second form or panel, loaded from **game_history.json** when the application starts. **[1 mark]**
 - Include a **New Game** button that resets both boards and starts a fresh game. **[0.5 mark]**
 - Include a **Quit** button that closes the application. **[0.5 mark]**
-

Part 5 - Additional Features (20%)

In Parts 1–4, you implemented the core game. In this part, select **four** additional features from the list below. Each feature is marked as **Medium** or **Hard**.

Gameplay Improvements:

- **Medium** - Difficulty levels. Add Easy, Medium and Hard computer opponent modes. On Easy, the computer picks questions randomly. On Medium, it eliminates at least half the remaining characters per question. On Hard, it always picks the question that eliminates the most characters.
- **Hard** - Hint system. Add a **Get Hint** button that suggests the best question for the human player to ask next, i.e., the one that would eliminate the most remaining characters. Each game allows a limited number of hints, e.g. three.
- **Medium** - Timer. Display a countdown timer per turn. If the human player does not ask a question before the timer expires, their turn is skipped.
- **Hard** - Custom character creator. Allow the player to add a new character with a custom name and attribute values via a form. Save custom characters to a JSON file and load them into the pool at the start of each game.

Statistics and History:

- **Medium** - Win/loss summary. Display a panel showing the player's total wins, total losses and win percentage calculated from the loaded game history.
- **Hard** - Turn-by-turn replay. After a game ends, allow the player to step through each turn that was taken, displaying which question was asked and how many characters were eliminated each turn.
- **Medium** - Leaderboard. Track and display the top five fastest wins, i.e., fewest turns, loaded from **game_history.json**.

- **Hard** - Per-attribute statistics. After each game, display which attributes were most and least useful, i.e., which eliminated the most and fewest characters on average, across the session.

UI and Accessibility:

- **Medium** - Character portraits. Display a simple drawn or coloured avatar for each character on the board generated from their attributes, e.g. hair colour, glasses, or hat, using **Graphics** or **Panel** controls.
 - **Medium** - Sound effects. Play a short sound when a character is eliminated, when a question is answered, and when the game ends.
 - **Hard** - Animated elimination. When a character is eliminated, animate a cross or fade effect over their card using a **Timer** control rather than instantly hiding them.
 - **Medium** - Colour theme selector. Allow the player to switch between at least three UI colour themes, e.g. Classic, Dark, or High Contrast, that update all form controls consistently.
-

Code Quality and Best Practices - Learning Outcome 2 (40%)

Naming Conventions (5%)

File, class, method, variable, constant and property names should be clear, descriptive and consistent across the codebase.

Comments (8%)

The code should be well-commented using the XML documentation format that explains complex logic and clarifies the purpose of classes, methods or complex sections.

Code Formatting (5%)

Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections.

Dead Code (5%)

The code should not contain unused or redundant files, classes, methods, variables, constants or properties. Keeping unnecessary code around can lead to confusion, clutter and potential bugs down the line.

Performance and Scalability (15%)

The code should be optimised for performance, using object-oriented programming principles, efficient algorithms and data structures, and error handling to minimise bottlenecks.

Here are some examples, but not a complete list, of error handling:

- When adding a custom character, the name should not be empty and should not already exist in the character pool
- Question selection should be validated - the player must select a question before submitting their turn

- Game history file errors, i.e., a missing or corrupted `game_history.json`, should be caught and handled gracefully, starting with an empty history if needed
- Final guess validation - the player must select a character name before submitting a guess

Class Diagram (2%)

Generate a class diagram using Visual Studio's built-in tools. This diagram should include all classes, methods and properties used in the application.

Additional Information

- You may add additional classes and methods beyond those specified.
- The computer player must make at least one guess per game. It should guess when only one character remains on its board.
- The human player may eliminate characters manually on their own board by clicking, independently of the question/answer system. This represents the player's own bookkeeping.
- Do not rewrite your Git history. It is important that the course lecturer can see how you worked on your assessment over time.